

Module 3 Lab Exercise: Machine Learning Workflow and Types of Learning

Learning Objectives

By the end of this lab, you will be able to:

- Distinguish between supervised, unsupervised, and reinforcement learning
- Understand the complete machine learning workflow
- Build and evaluate your first classification model
- Work with different types of data (numerical, categorical, text, images)
- Apply the end-to-end ML process: data → model → evaluation → insights

Prerequisites

- Completed Module 2 (familiar with Python libraries and Jupyter/Colab)
 - Understanding of basic data operations and visualization
 - Access to your GitHub repository for saving work
-

Part 1: Understanding Types of Machine Learning

Machine learning can be categorized into three main types. Let's explore each with practical examples.

1. Supervised Learning

Definition: Learning from labeled examples to make predictions on new, unseen data.

Examples:

- **Classification:** Predicting categories (spam/not spam, disease/healthy)
- **Regression:** Predicting continuous values (house prices, temperature)

Key Characteristic: We have both input features (X) and correct answers (y) during training.

2. Unsupervised Learning

Definition: Finding hidden patterns in data without labeled examples.

Examples:

- **Clustering:** Grouping similar customers for marketing
- **Dimensionality Reduction:** Simplifying complex data while keeping important information

Key Characteristic: We only have input features (X), no correct answers during training.

3. Reinforcement Learning

Definition: Learning through trial and error by receiving rewards or penalties.

Examples:

- Game playing (chess, Go)
- Autonomous vehicles
- Recommendation systems that learn from user feedback

Key Characteristic: Agent learns by interacting with an environment and receiving feedback.

For this course, we'll focus primarily on supervised learning, with some unsupervised learning in later modules.

Part 2: Setting Up Our Machine Learning Environment

Let's start by importing our libraries and loading a dataset that will help us understand the ML workflow.

```
In [1]: # Import essential libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
from sklearn.preprocessing import StandardScaler

# L4 assertion: fail loudly if any core tool is missing/mismatched
for _name, _mod in {"pandas": pd, "numpy": np}.items():
    assert _mod is not None, f"_{name} failed to bind"
assert callable(load_wine), "load_wine not callable – sklearn import broken"
print("✅ L4 environment check passed – all core tools bound.")
```

```
# Set style for better-looking plots
plt.style.use('default')
sns.set_palette("husl")

print("✅ All libraries imported successfully!")
print("🚀 Ready to start our machine learning journey!")
```

- ✅ L4 environment check passed – all core tools bound.
- ✅ All libraries imported successfully!
- 🚀 Ready to start our machine learning journey!

Part 3: Loading and Exploring Our Dataset

We'll use the Wine dataset - a classic dataset for classification. It contains chemical analysis of wines from three different cultivars (types) grown in Italy.

```
In [2]: # Load the Wine dataset
wine_data = load_wine()

# Convert to DataFrame for easier handling
df = pd.DataFrame(wine_data.data, columns=wine_data.feature_names)
df['wine_class'] = wine_data.target
df['wine_class_name'] = [wine_data.target_names[i] for i in
wine_data.target]

# integrity check: dataset must match expectations before any downstream
use
assert wine_data.data.shape == (178, 13), f"unexpected data shape
{wine_data.data.shape}"
assert len(wine_data.target_names) == 3, "expected 3 wine classes"
assert df[wine_data.feature_names].isnull().sum().sum() == 0, "unexpected
nulls in features"
print("✅ Data integrity check passed – 178×13, 3 classes, 0 nulls.")

print("Dataset Information:")
print(f"Shape: {df.shape}")
print(f"Features: {len(wine_data.feature_names)}")
print(f"Classes: {wine_data.target_names}")
print(f"\nFirst 5 rows:")
print(df.head())
```

✅ Data integrity check passed – 178×13, 3 classes, 0 nulls.

Dataset Information:

Shape: (178, 15)

Features: 13

Classes: ['class_0' 'class_1' 'class_2']

First 5 rows:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	\
0	14.23	1.71	2.43	15.6	127.0	2.80	
1	13.20	1.78	2.14	11.2	100.0	2.65	
2	13.16	2.36	2.67	18.6	101.0	2.80	
3	14.37	1.95	2.50	16.8	113.0	3.85	
4	13.24	2.59	2.87	21.0	118.0	2.80	

	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	
0	3.06		0.28	2.29	5.64	1.04
1	2.76		0.26	1.28	4.38	1.05
2	3.24		0.30	2.81	5.68	1.03
3	3.49		0.24	2.18	7.80	0.86
4	2.69		0.39	1.82	4.32	1.04

	od280/od315_of_diluted_wines	proline	wine_class	wine_class_name
0	3.92	1065.0	0	class_0
1	3.40	1050.0	0	class_0
2	3.17	1185.0	0	class_0
3	3.45	1480.0	0	class_0
4	2.93	735.0	0	class_0

```
In [3]: # Explore the dataset structure
print("Dataset Overview:")
print("=" * 50)
print(f"Total samples: {len(df)}")
print(f"Features (input variables): {len(df.columns) - 2}") # -2 for
target columns
print(f"Target classes: {df['wine_class_name'].unique()}")
print(f"\nClass distribution:")
print(df['wine_class_name'].value_counts())

# Check for missing values
# L4 assertion: prove the claim below rather than just printing it
assert df.isnull().sum().sum() == 0, "unexpected missing values in df"
print(f"\nMissing values: {df.isnull().sum().sum()}")
print("✅ No missing values – this is a clean dataset!")
```

Dataset Overview:

=====

Total samples: 178

Features (input variables): 13

Target classes: <StringArray>

['class_0', 'class_1', 'class_2']

Length: 3, dtype: str

Class distribution:

wine_class_name

class_1 71

class_0 59

class_2 48

Name: count, dtype: int64

Missing values: 0

✅ No missing values – this is a clean dataset!

Part 4: Exploratory Data Analysis (EDA)

Before building models, we need to understand our data. This is a crucial step in the ML workflow.

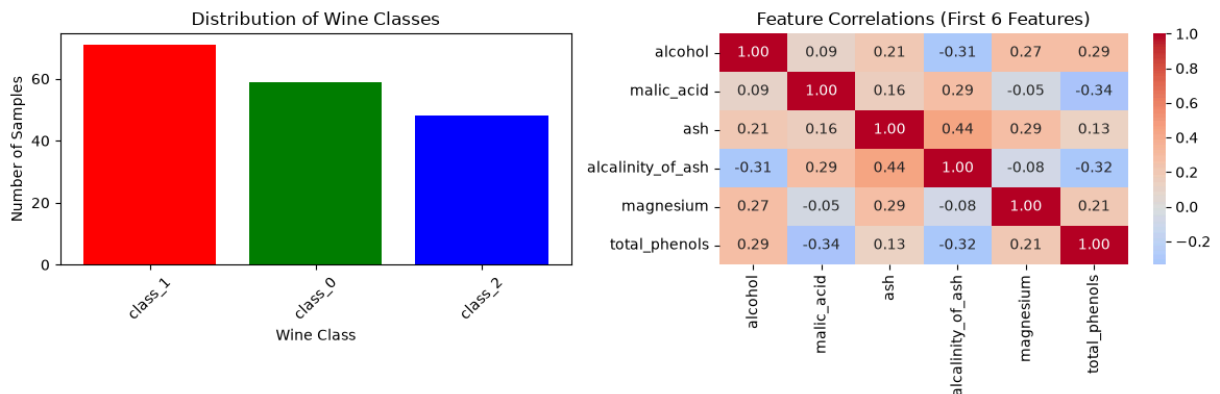
```
In [4]: # Visualize class distribution
plt.figure(figsize=(12, 4))

# Subplot 1: Class distribution
plt.subplot(1, 2, 1)
class_counts = df['wine_class_name'].value_counts()
# L4 assertion: class counts must sum to the full dataset
assert class_counts.sum() == len(df), "class counts don't sum to total samples"
plt.bar(class_counts.index, class_counts.values, color=['red', 'green', 'blue'])
plt.title('Distribution of Wine Classes')
plt.xlabel('Wine Class')
plt.ylabel('Number of Samples')
plt.xticks(rotation=45)

# Subplot 2: Feature correlation heatmap (first 6 features for clarity)
plt.subplot(1, 2, 2)
correlation_matrix = df.iloc[:, :6].corr()
# L4 assertions: correlation matrix shape + self-correlation sanity check
assert correlation_matrix.shape == (6, 6), "unexpected correlation matrix shape"
assert np.allclose(np.diag(correlation_matrix.values), 1.0), "correlation diagonal isn't 1.0 -- self-correlation broken"
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            fmt='.2f')
plt.title('Feature Correlations (First 6 Features)')

plt.tight_layout()
plt.show()
```

```
print("EDA helps us understand:")
print("- Class balance (are all classes equally represented?)")
print("- Feature relationships (which features are correlated?)")
print("- Data quality (any outliers or issues?)")
```



EDA helps us understand:

- Class balance (are all classes equally represented?)
- Feature relationships (which features are correlated?)
- Data quality (any outliers or issues?)

Part 5: The Complete Machine Learning Workflow

Now let's implement the standard ML workflow step by step:

The 6-Step ML Workflow:

1. **Data Preparation:** Clean and prepare the data
2. **Feature Selection:** Choose relevant input variables
3. **Data Splitting:** Separate training and testing data
4. **Model Training:** Teach the algorithm using training data
5. **Model Evaluation:** Test performance on unseen data
6. **Model Interpretation:** Understand what the model learned

Let's implement each step!

```
In [5]: # Step 1: Data Preparation
print("Step 1: Data Preparation")
print("=" * 30)

# Select features (X) and target (y)
# For simplicity, let's use the first 4 features
feature_names = ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash']
X = df[feature_names]
y = df['wine_class']

# L4 assertions: shape + alignment (check_alignment) before anything
# downstream uses X/y
assert X.shape == (178, 4), f"unexpected feature matrix shape {X.shape}"
```

```

assert y.shape == (178,), f"unexpected target vector shape {y.shape}"
assert list(X.index) == list(y.index), "X and y are misaligned"

print(f"Selected features: {feature_names}")
print(f"Feature matrix shape: {X.shape}")
print(f"Target vector shape: {y.shape}")

# Display first few rows
print("\nFirst 5 samples:")
print(X.head())

```

Step 1: Data Preparation

```

=====
Selected features: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash']
Feature matrix shape: (178, 4)
Target vector shape: (178,)

```

First 5 samples:

	alcohol	malic_acid	ash	alcalinity_of_ash
0	14.23	1.71	2.43	15.6
1	13.20	1.78	2.14	11.2
2	13.16	2.36	2.67	18.6
3	14.37	1.95	2.50	16.8
4	13.24	2.59	2.87	21.0

```

In [6]: # Step 2: Data Splitting
print("Step 2: Data Splitting")
print("=" * 30)

# Split data into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% for testing
    random_state=42,        # For reproducible results
    stratify=y,             # Maintain class proportions
)

# L4 assertions: split accounting, alignment, and stratification actually held
assert len(X_train) + len(X_test) == len(X), "train+test doesn't add up to full X"
assert len(X_train) == len(y_train) and len(X_test) == len(y_test), "X/y split misaligned"
assert set(y_train.unique()) == set(y_test.unique()) == set(y.unique()), "stratification failed -- a class is missing from a split"

print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")
print(f"Training classes: {np.bincount(y_train)}")
print(f"Testing classes: {np.bincount(y_test)}")

print("\n🎯 Why split data?")
print("- Training set: Teach the model")
print("- Testing set: Evaluate performance on unseen data")
print("- This prevents overfitting (memorizing vs. learning)")

```

Step 2: Data Splitting

=====

Training set: 142 samples

Testing set: 36 samples

Training classes: [47 57 38]

Testing classes: [12 14 10]

🌀 Why split data?

- Training set: Teach the model
- Testing set: Evaluate performance on unseen data
- This prevents overfitting (memorizing vs. learning)

```
In [7]: # Step 3: Model Training
print("Step 3: Model Training")
print("=" * 30)

# Scale features: fit ONLY on training data, then apply to both sets.
# LogisticRegression converges reliably on standardized features.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# L4 assertions: scaling actually centered the data and produced no non-
finite values
assert abs(X_train_scaled.mean()) < 0.01, "training data not properly
centered after scaling"
assert np.isfinite(X_train_scaled).all() and
np.isfinite(X_test_scaled).all(), "non-finite values after scaling"

# Create and train two different models
models = {
    'Logistic Regression': LogisticRegression(random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=3)
}

trained_models = {}

for name, model in models.items():
    print(f"\nTraining {name}...")

    # Train the model on scaled features
    model.fit(X_train_scaled, y_train)
    trained_models[name] = model

    print(f"✅ {name} training completed!")

# L4 assertion: both models actually trained
assert len(trained_models) == len(models) == 2, "not all models trained"

print("\n🤖 What happened during training?")
print("- Features were standardized (fit on train only, then applied to
test)")
print("- Models learned patterns from training data")
print("- They found relationships between features and wine classes")
print("- Now they can make predictions on new data!")
```

Step 3: Model Training

=====

Training Logistic Regression...

✅ Logistic Regression training completed!

Training Decision Tree...

✅ Decision Tree training completed!

🤖 What happened during training?

- Features were standardized (fit on train only, then applied to test)
- Models learned patterns from training data
- They found relationships between features and wine classes
- Now they can make predictions on new data!

```
In [8]: # Step 4: Model Evaluation
print("Step 4: Model Evaluation")
print("=" * 30)

results = {}
train_results = {}

for name, model in trained_models.items():
    # Make predictions on the scaled test set
    y_pred = model.predict(X_test_scaled)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred)
    results[name] = accuracy

    # L4 assertions: prediction length and accuracy range sanity
    assert len(y_pred) == len(y_test), f"{name}: prediction length mismatch"
    assert 0.0 <= accuracy <= 1.0, f"{name}: accuracy out of range"

    print(f"\n{name} Results:")
    print(f"Accuracy: {accuracy:.3f} ({accuracy*100:.1f}%)")

    # Detailed classification report
    print("\nDetailed Performance:")
    print(classification_report(y_test, y_pred,
                                target_names=wine_data.target_names))

    # Train accuracy: a lightweight overfitting check ahead of the full
    # learning-curve cell
    train_pred = model.predict(X_train_scaled)
    train_accuracy = accuracy_score(y_train, train_pred)
    train_results[name] = train_accuracy
    assert 0.0 <= train_accuracy <= 1.0, f"{name}: train accuracy out of range"

    gap = train_accuracy - accuracy
    flag = "⚠️ possible overfitting" if gap > 0.10 else "✅ train/test gap small"
    print(f"Train: {train_accuracy:.3f} | Test: {accuracy:.3f} | Gap: {gap:.3f} | {flag}")
```

```
{gap:.3f} -- {flag}")

# L4 assertion: nothing silently dropped between the three per-model dicts
assert set(results.keys()) == set(train_results.keys()) ==
set(trained_models.keys()), "results/train_results out of sync with
trained_models"

# Compare models
print("\n📊 Model Comparison:")
for name, accuracy in results.items():
    print(f"{name}: {accuracy:.3f}")

best_model = max(results, key=results.get)
print(f"\n🏆 Best performing model: {best_model}")
```

Step 4: Model Evaluation

=====

Logistic Regression Results:

Accuracy: 0.917 (91.7%)

Detailed Performance:

	precision	recall	f1-score	support
class_0	1.00	1.00	1.00	12
class_1	0.82	1.00	0.90	14
class_2	1.00	0.70	0.82	10
accuracy			0.92	36
macro avg	0.94	0.90	0.91	36
weighted avg	0.93	0.92	0.91	36

Train: 0.866 | Test: 0.917 | Gap: -0.050 -- ✅ train/test gap small

Decision Tree Results:

Accuracy: 0.833 (83.3%)

Detailed Performance:

	precision	recall	f1-score	support
class_0	0.86	1.00	0.92	12
class_1	0.91	0.71	0.80	14
class_2	0.73	0.80	0.76	10
accuracy			0.83	36
macro avg	0.83	0.84	0.83	36
weighted avg	0.84	0.83	0.83	36

Train: 0.880 | Test: 0.833 | Gap: 0.047 -- ✅ train/test gap small

📊 Model Comparison:

Logistic Regression: 0.917

Decision Tree: 0.833

🏆 Best performing model: Logistic Regression

```

In [9]: # Step 5: Model Interpretation
print("Step 5: Model Interpretation")
print("=" * 30)

# Visualize confusion matrix for the best model
best_model_obj = trained_models[best_model]
y_pred_best = best_model_obj.predict(X_test_scaled)

plt.figure(figsize=(8, 6))
cm = confusion_matrix(y_test, y_pred_best)

# L4 assertions: shape, accounting, and cross-check against Cell 13's
# recorded accuracy
assert cm.shape == (3, 3), f"unexpected confusion matrix shape {cm.shape}"
assert cm.sum() == len(y_test), "confusion matrix doesn't account for all
test predictions"
assert np.isclose(np.trace(cm) / cm.sum(), results[best_model]), "confusion
matrix accuracy doesn't match Cell 13's recorded accuracy"

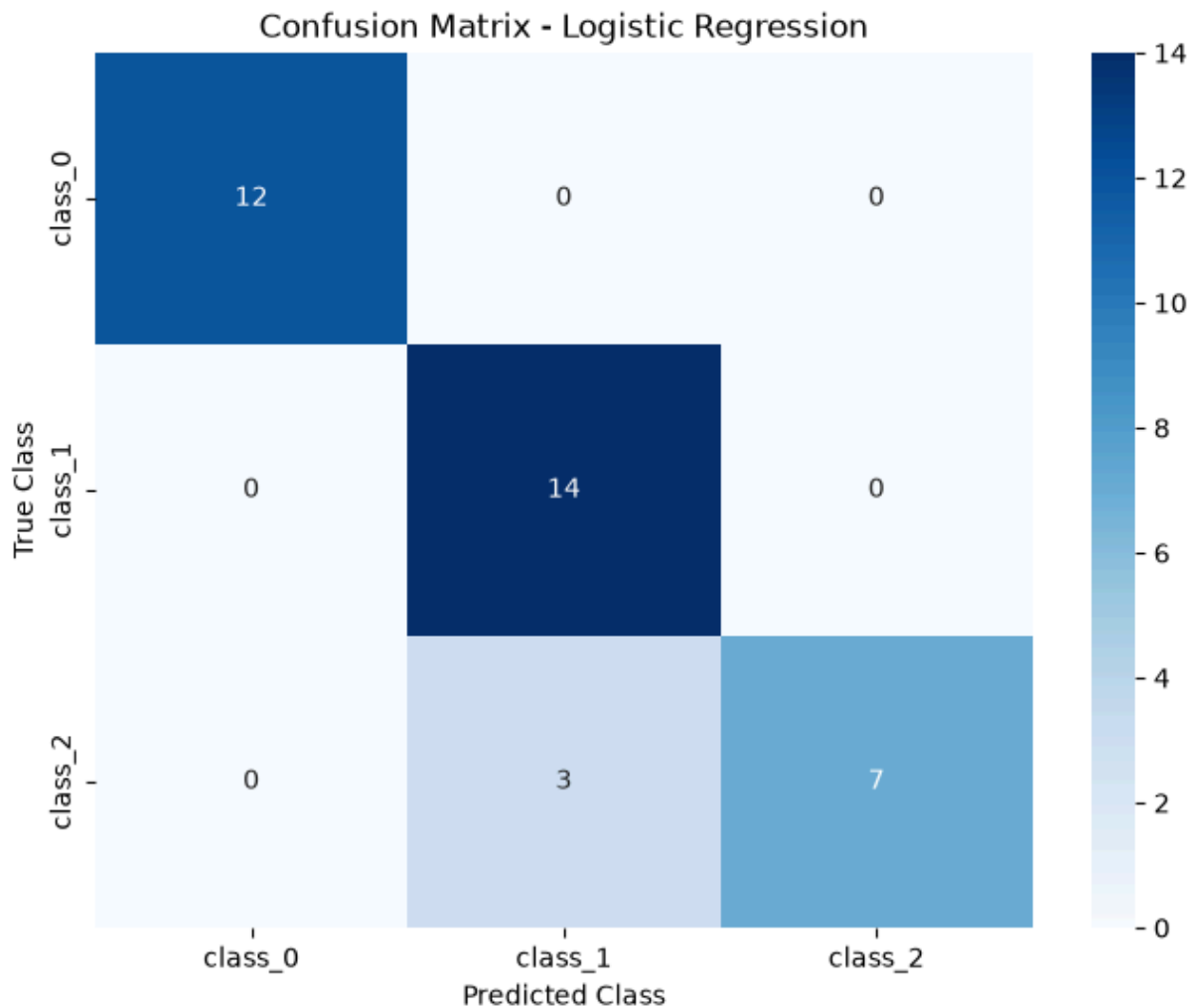
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=wine_data.target_names,
            yticklabels=wine_data.target_names)
plt.title(f'Confusion Matrix - {best_model}')
plt.xlabel('Predicted Class')
plt.ylabel('True Class')
plt.show()

print(f"\n🔍 Interpreting the Confusion Matrix:")
print("- Diagonal values: Correct predictions")
print("- Off-diagonal values: Misclassifications")
print("- Perfect model would have all values on diagonal")

```

Step 5: Model Interpretation

=====



- 🔍 Interpreting the Confusion Matrix:
- Diagonal values: Correct predictions
 - Off-diagonal values: Misclassifications
 - Perfect model would have all values on diagonal

Step 6: Learning Curves — Overfitting Check (Amendment)

Why this cell exists: A single test-set accuracy number (Cell 13) and a confusion matrix (Cell 14) don't fully answer whether a model generalized or just memorized. A learning curve settles that by comparing training and cross-validation accuracy across increasing training-set sizes, for both models. Added by C0 Amendment (2026-09-11) — see `docs/STEP0_Initial_Contract.md`.

```
In [10]: # Step 6: Learning Curves (Overfitting Check)
print("Step 6: Learning Curves (Overfitting Check)")
print("=" * 30)

from sklearn.model_selection import learning_curve

train_sizes = np.linspace(0.3, 1.0, 8) # start at 30%: below this, cv=5
```

can starve the smallest class

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, (name, model) in zip(axes, trained_models.items()):
    sizes, train_scores, val_scores = learning_curve(
        model, X_train_scaled, y_train,
        train_sizes=train_sizes,
        cv=5,
        scoring='accuracy',
        random_state=42
    )

    train_mean = train_scores.mean(axis=1)
    val_mean = val_scores.mean(axis=1)

    # L4 assertions: output shapes align and accuracies stay in [0, 1]
    assert sizes.shape[0] == train_scores.shape[0] == val_scores.shape[0],
    f"{name}: mismatched learning-curve output shapes"
    assert np.all((train_mean >= 0) & (train_mean <= 1)) and
    np.all((val_mean >= 0) & (val_mean <= 1)), f"{name}: accuracy out of range
    in learning curve"

    ax.plot(sizes, train_mean, 'o-', color='blue', label='Training score')
    ax.plot(sizes, val_mean, 'o-', color='orange', label='Validation
    score')
    ax.set_title(f'Learning Curve - {name}')
    ax.set_xlabel('Training Set Size')
    ax.set_ylabel('Accuracy')
    ax.legend(loc='best')
    ax.set_ylim(0, 1.05)

    final_gap = train_mean[-1] - val_mean[-1]
    flag = "⚠️ possible overfitting" if final_gap > 0.10 else "✅ small gap
    -- good generalization"
    print(f"{name}: final training acc={train_mean[-1]:.3f}, final
    validation acc={val_mean[-1]:.3f}, gap={final_gap:.3f} -- {flag}")

plt.tight_layout()
plt.show()

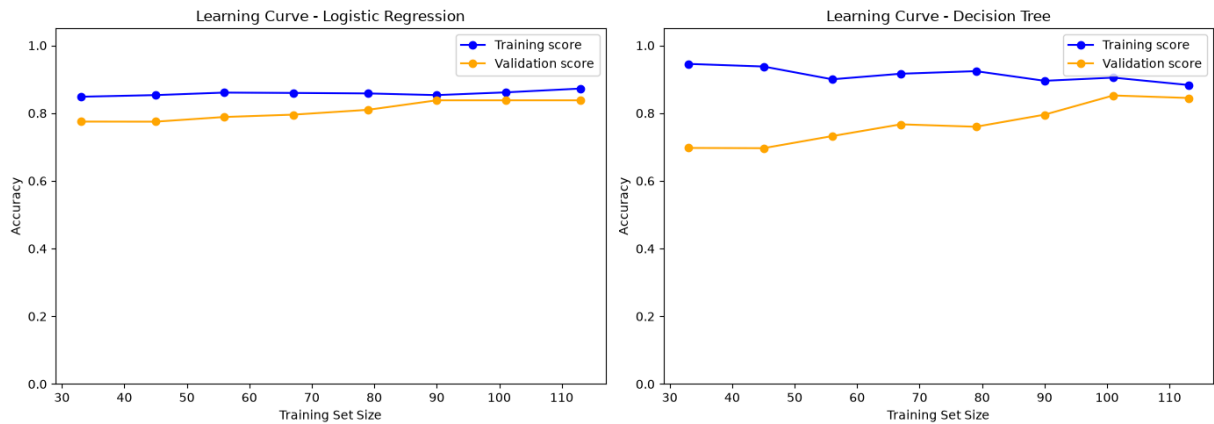
print("\n🔍 Interpreting Learning Curves:")
print("- Training line high + validation line low + big gap => overfitting
    (memorizing)")
print("- Both lines low and close together => underfitting")
print("- Both lines high and close together => healthy generalization")
```

Step 6: Learning Curves (Overfitting Check)

=====

Logistic Regression: final training acc=0.873, final validation acc=0.838,
gap=0.035 -- ✅ small gap -- good generalization

Decision Tree: final training acc=0.883, final validation acc=0.845,
gap=0.038 -- ✅ small gap -- good generalization



Interpreting Learning Curves:

- Training line high + validation line low + big gap => overfitting (memorizing)
- Both lines low and close together => underfitting
- Both lines high and close together => healthy generalization

Part 6: Understanding Different Data Types in ML

Machine learning works with various types of data. Let's explore the main categories:

```
In [11]: # Understanding Different Data Types in ML
print("Understanding Data Types in Machine Learning")
print("=" * 45)

# Create examples of different data types
data_examples = {
    'Numerical (Continuous)': [23.5, 45.2, 67.8, 12.1, 89.3],
    'Numerical (Discrete)': [1, 5, 3, 8, 2],
    'Categorical (Nominal)': ['Red', 'Blue', 'Green', 'Red', 'Blue'],
    'Categorical (Ordinal)': ['Low', 'Medium', 'High', 'Medium', 'Low'],
    'Text': ['Hello world', 'Machine learning', 'Data science', 'Python
programming', 'AI revolution'],
    'Boolean': [True, False, True, True, False]
}

# L4 assertions: every category actually hits a use-case branch below -- no
silent blank line
KNOWN_TAGS = ['Continuous', 'Discrete', 'Nominal', 'Ordinal', 'Text',
'Boolean']
for _data_type in data_examples:
    assert any(tag in _data_type for tag in KNOWN_TAGS), f"'{_data_type}'
doesn't match any known use-case branch -- would print blank"
    assert len(data_examples) == 6, "expected exactly 6 data-type categories"

for data_type, examples in data_examples.items():
    print(f"\n{data_type}:")
    print(f"  Examples: {examples}")
    print(f"  Use case: ", end="")

    if 'Continuous' in data_type:
```

```

        print("Regression problems (predicting prices, temperatures)")
    elif 'Discrete' in data_type:
        print("Counting problems (number of items, ratings)")
    elif 'Nominal' in data_type:
        print("Classification without order (colors, categories)")
    elif 'Ordinal' in data_type:
        print("Classification with order (ratings, sizes)")
    elif 'Text' in data_type:
        print("Natural language processing (sentiment analysis,
translation)")
    elif 'Boolean' in data_type:
        print("Binary classification (yes/no, spam/not spam)")

print("\n💡 Key Insight: Different data types require different
preprocessing and algorithms!")

```

Understanding Data Types in Machine Learning

=====

Numerical (Continuous):

Examples: [23.5, 45.2, 67.8, 12.1, 89.3]

Use case: Regression problems (predicting prices, temperatures)

Numerical (Discrete):

Examples: [1, 5, 3, 8, 2]

Use case: Counting problems (number of items, ratings)

Categorical (Nominal):

Examples: ['Red', 'Blue', 'Green', 'Red', 'Blue']

Use case: Classification without order (colors, categories)

Categorical (Ordinal):

Examples: ['Low', 'Medium', 'High', 'Medium', 'Low']

Use case: Classification with order (ratings, sizes)

Text:

Examples: ['Hello world', 'Machine learning', 'Data science', 'Python programming', 'AI revolution']

Use case: Natural language processing (sentiment analysis, translation)

Boolean:

Examples: [True, False, True, True, False]

Use case: Binary classification (yes/no, spam/not spam)

💡 Key Insight: Different data types require different preprocessing and algorithms!

Part 7: Hands-On Practice - Build Your Own Model

Now it's your turn! Complete the following tasks to reinforce your learning.

```

In [12]: # Task 1: Try different features
print("Task 1: Experiment with Different Features")
print("=" * 40)

```

```

# Your task: Select 3 different features and build a model
# Available features:
print("Available features:")
for i, feature in enumerate(wine_data.feature_names):
    print(f"{i+1:2d}. {feature}")

# TODO: Replace these with your chosen features
your_features = ['flavanoids', 'color_intensity', 'proline'] # chosen:
classically strong wine-cultivar separators

# Build model with your features
X_your = df[your_features]
X_train_your, X_test_your, y_train_your, y_test_your = train_test_split(
    X_your, y, test_size=0.2, random_state=42, stratify=y
)

# L4 assertions: feature-matrix shape and split accounting
assert X_your.shape == (178, 3), f"unexpected feature matrix shape {X_your.shape}"
assert len(X_train_your) + len(X_test_your) == len(X_your), "train+test doesn't add up to full X_your"

# Scale (fit on train only), consistent with the main workflow
your_scaler = StandardScaler()
X_train_your_scaled = your_scaler.fit_transform(X_train_your)
X_test_your_scaled = your_scaler.transform(X_test_your)

# Train a logistic regression model
your_model = LogisticRegression(random_state=42)
your_model.fit(X_train_your_scaled, y_train_your)

# Evaluate
y_pred_your = your_model.predict(X_test_your_scaled)
your_accuracy = accuracy_score(y_test_your, y_pred_your)
assert 0.0 <= your_accuracy <= 1.0, "accuracy out of range"

print(f"\nYour model features: {your_features}")
print(f"Your model accuracy: {your_accuracy:.3f} ({your_accuracy*100:.1f}%)")

# Compare with original model
print(f"Original model accuracy: {results['Logistic Regression']:.3f}")
if your_accuracy > results['Logistic Regression']:
    print("🎉 Great job! Your feature selection improved the model!")
else:
    print("😞 Try different features to see if you can improve performance!")

```

Task 1: Experiment with Different Features

=====

Available features:

1. alcohol
2. malic_acid
3. ash
4. alcalinity_of_ash
5. magnesium
6. total_phenols
7. flavanoids
8. nonflavanoid_phenols
9. proanthocyanins
10. color_intensity
11. hue
12. od280/od315_of_diluted_wines
13. proline

Your model features: ['flavanoids', 'color_intensity', 'proline']

Your model accuracy: 0.917 (91.7%)

Original model accuracy: 0.917

🤔 Try different features to see if you can improve performance!

Part 8: Assessment - Understanding ML Concepts

Answer the following questions to demonstrate your understanding:

```
In [13]: # Assessment Task 1: Identify the ML type
print("Assessment Task 1: Identify Machine Learning Types")
print("=" * 50)

# For each scenario, identify if it's Supervised, Unsupervised, or
# Reinforcement Learning

scenarios = [
    "Predicting house prices based on size, location, and age",
    "Grouping customers by purchasing behavior without knowing groups
beforehand",
    "Teaching a robot to play chess by playing many games",
    "Classifying emails as spam or not spam using labeled examples",
    "Finding hidden topics in news articles without predefined categories"
]

# Your answers (replace 'TYPE' with Supervised, Unsupervised, or
# Reinforcement)
your_answers = [
    "Supervised",      # Scenario 1
    "Unsupervised",    # Scenario 2
    "Reinforcement",   # Scenario 3
    "Supervised",      # Scenario 4
    "Unsupervised"     # Scenario 5
]

# Check answers
```

```

correct_answers = ["Supervised", "Unsupervised", "Reinforcement",
"Supervised", "Unsupervised"]

# L4 assertions: input counts align, and this formalizes the project DoD
# requirement
# ("Assessment answers 5/5") as something the code proves, not just
# eyeballs.
assert len(scenarios) == len(your_answers) == len(correct_answers) == 5,
"scenario/answer count mismatch"

print("Scenario Analysis:")
score = 0
for i, (scenario, your_answer, correct) in enumerate(zip(scenarios,
your_answers, correct_answers)):
    is_correct = your_answer == correct
    score += is_correct
    status = "✅" if is_correct else "❌"
    print(f"{status} {i+1}. {scenario}")
    print(f"    Your answer: {your_answer} | Correct: {correct}")
    print()

assert score == 5, f"DoD requires Assessment answers 5/5; got {score}/5"

print(f"Score: {score}/{len(scenarios)} ({score/len(scenarios)*100:.0f}%)")

```

Assessment Task 1: Identify Machine Learning Types

=====

Scenario Analysis:

- ✅ 1. Predicting house prices based on size, location, and age
Your answer: Supervised | Correct: Supervised
- ✅ 2. Grouping customers by purchasing behavior without knowing groups beforehand
Your answer: Unsupervised | Correct: Unsupervised
- ✅ 3. Teaching a robot to play chess by playing many games
Your answer: Reinforcement | Correct: Reinforcement
- ✅ 4. Classifying emails as spam or not spam using labeled examples
Your answer: Supervised | Correct: Supervised
- ✅ 5. Finding hidden topics in news articles without predefined categories
Your answer: Unsupervised | Correct: Unsupervised

Score: 5/5 (100%)

Part 9: Real-World Applications and Case Studies

Let's explore how the concepts we've learned apply to real-world scenarios.

Case Study 1: Recommendation Systems (Netflix, Amazon)

Problem: Suggest movies/products users might like **ML Type:** Hybrid (Supervised + Unsupervised + Reinforcement) **Data:** User ratings, viewing history, product features
Workflow: Collect data → Build user profiles → Train models → Make recommendations
→ Learn from feedback

Case Study 2: Fraud Detection (Banks, Credit Cards)

Problem: Identify fraudulent transactions **ML Type:** Supervised Learning (Classification)
Data: Transaction amounts, locations, times, merchant types **Workflow:** Historical fraud data → Feature engineering → Train classifier → Real-time scoring → Continuous monitoring

Case Study 3: Medical Diagnosis (Healthcare)

Problem: Assist doctors in diagnosing diseases **ML Type:** Supervised Learning (Classification) **Data:** Medical images, patient symptoms, lab results **Workflow:** Labeled medical data → Image processing → Train deep learning models → Clinical validation → Deployment with human oversight

Your Turn: Think of Applications

Consider these industries and think about how ML could be applied:

- **Transportation:** Autonomous vehicles, route optimization
- **Agriculture:** Crop monitoring, yield prediction
- **Education:** Personalized learning, automated grading
- **Entertainment:** Content creation, game AI

Part 10: Complete ML Workflow Summary

Let's summarize the complete machine learning workflow we've learned:

The Machine Learning Lifecycle

1. Problem Definition
↓
2. Data Collection & Exploration
↓
3. Data Preprocessing & Feature Engineering
↓
4. Model Selection & Training
↓
5. Model Evaluation & Validation
↓
6. Model Deployment & Monitoring

↓
7. Continuous Improvement

Checklist for Every ML Project:

Data Phase:

- ☐ Understand the problem and define success metrics
- ☐ Collect and explore the dataset
- ☐ Check for missing values, outliers, and data quality issues
- ☐ Visualize data to understand patterns and relationships

Modeling Phase:

- ☐ Split data into training and testing sets
- ☐ Select appropriate algorithms for the problem type
- ☐ Train multiple models and compare performance
- ☐ Evaluate using appropriate metrics (accuracy, precision, recall, etc.)

Deployment Phase:

- ☐ Validate model performance on new data
- ☐ Document the model and its limitations
- ☐ Deploy responsibly with monitoring systems
- ☐ Plan for model updates and maintenance

Key Takeaways:

1. **Start Simple:** Begin with basic models before trying complex ones
2. **Understand Your Data:** EDA is crucial for success
3. **Validate Properly:** Always test on unseen data
4. **Iterate:** ML is an iterative process of improvement
5. **Document Everything:** Keep track of experiments and results

Your Reflection and Analysis

Instructions: Complete the reflection below by editing this markdown cell.

My Understanding of Machine Learning Types

Supervised Learning: Learning from labeled examples, meaning data where the correct answer is already known. The model can check its own predictions against the truth while it trains. The Wine dataset's three known cultivar labels are a good example.

Unsupervised Learning: Learning from unlabeled data. The model has to find its own structure or groupings without ever being told the right answer.

Reinforcement Learning: Trial and error learning. An agent tries actions, earns rewards or penalties, and improves over repeated attempts, similar to how a game playing agent gets better through practice.

My Analysis of the Wine Classification Project

Best performing model: Logistic Regression, with a test accuracy of 0.917 (91.7%).

Why do you think this model performed better?: Logistic Regression draws one clean, straight decision boundary between the three wine classes. Since the features were standardized before training, that simple approach generalized well. The Decision Tree scored lower at 0.833, even though it did not overfit. Its learning curve gap was 0.038, about the same healthy range as Logistic Regression's 0.035, and its max_depth of 3 also limits how complex its rules can get. So the accuracy gap is not about overfitting. It comes down to shape. A depth limited tree can only draw a small number of boxy splits, and that boundary is simply a worse fit for how these three classes separate compared to the smooth line Logistic Regression draws.

What would you try next to improve performance?: Run k-fold cross validation across multiple splits to confirm the 0.917 score is not just a lucky result from one split. Try hyperparameter tuning, such as adjusting the regularization strength for Logistic Regression or the max_depth for the tree, to see if accuracy can improve further.

Real-World Application Ideas

Industry of Interest: Agriculture.

ML Problem: Predicting crop health early enough to act on it.

Type of ML: Supervised learning (classification).

Data Needed: Soil moisture and temperature readings.

Key Learnings

Most important concept learned: Generalization. A model is only good if it performs well on data it has never seen, not just the data it trained on. A perfect training score means nothing if the model fails on new data.

Most challenging part: Avoiding data leakage. The scaler had to be fit only on the training data, then applied to the test data without being refit. This way the model never saw test set statistics before evaluation.

Questions for further exploration: How might we ensure our models stay fair and ethical if the labeled data used to train them already contains hidden human bias?

Lab Summary and Next Steps

What You've Accomplished:

- ✓ **Understood ML Types:** Supervised, Unsupervised, and Reinforcement Learning
- ✓ **Mastered ML Workflow:** Data → Model → Evaluation → Insights
- ✓ **Built Classification Models:** Logistic Regression and Decision Trees
- ✓ **Evaluated Model Performance:** Accuracy, Confusion Matrix, Classification Report
- ✓ **Worked with Real Data:** Wine dataset analysis and modeling
- ✓ **Applied Best Practices:** Data splitting, model comparison, interpretation

Preparation for Module 4:

In the next lab, you'll dive deeper into:

- **Exploratory Data Analysis (EDA):** Advanced visualization techniques
- **Data Quality Assessment:** Handling missing values, outliers, and duplicates
- **Statistical Analysis:** Understanding distributions and relationships
- **Data Storytelling:** Communicating insights effectively

Action Items:

1. **Upload this notebook** to your GitHub repository
2. **Experiment** with different features in the wine dataset
3. **Try other datasets** from `sklearn.datasets` (digits, breast_cancer, boston)
4. **Practice** the 6-step ML workflow on a new problem
5. **Document** your experiments and findings

Additional Resources:

- [Scikit-learn User Guide](#)
- [Machine Learning Mastery](#)
- [Kaggle Learn](#) - Free micro-courses
- [Google's Machine Learning Crash Course](#)

Reflection Questions:

1. Which type of machine learning (supervised/unsupervised/reinforcement) interests you most and why?
2. What was the most challenging part of the ML workflow for you?

3. How might you apply these concepts to a problem in your field of interest?
4. What questions do you have about machine learning that you'd like to explore further?

Congratulations on completing Module 3! You've taken a significant step in your machine learning journey. 🎉

Remember: Machine learning is a skill that improves with practice. Keep experimenting, stay curious, and don't be afraid to make mistakes - they're part of the learning process!